

Dr. Isah Charles Saidu

Lead Instructor

Liatris Systems

AI & Technology Solutions

Session Duration: 1 Hour 30 minutes

July 5, 2026



Liatris Systems



In partnership with the Independent National Electoral Commission

Session Roadmap

Part 1: Deep Learning

-  What is Deep Learning?
-  Neural Network Architecture
-  Training Deep Networks

Part 2: Model Development

-  PyTorch Fundamentals
-  Building Neural Networks
-  Transfer Learning

Part 3: Deployment

-  Deployment Options
-  FastAPI for ML
-  Containerization

Part 4: INEC Applications

-  Election Use Cases
-  Practical Implementation

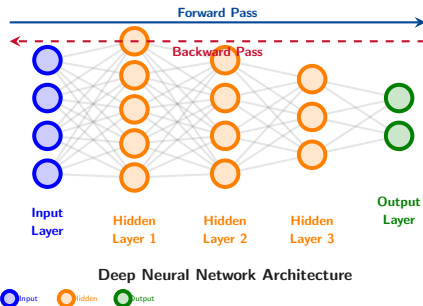
What is Deep Learning?

What is Deep Learning?

Deep Learning is a subset of Machine Learning that uses **neural networks with multiple layers** to automatically learn hierarchical representations of data.

Key Characteristics

- Automatic Feature Extraction
- Hierarchical Learning
- End-to-End Learning
- Handles Unstructured Data
- Requires Large Datasets



Why Deep Learning Now?

Three Key Drivers

1. Big Data

- Massive digital data
- Labeled datasets available

2. Computing Power

- GPUs and TPUs
- Cloud computing

3. Algorithm Advances

- Backpropagation
- Activation functions
- Dropout, BatchNorm

Deep Learning vs Traditional ML

• Feature Engineering

- ML: Manual
- DL: Automatic

• Data Requirements

- ML: Small datasets
- DL: Large datasets

• Interpretability

- ML: High
- DL: Low

• Performance

- ML: Good
- DL: State-of-the-art

Neural Network Architecture

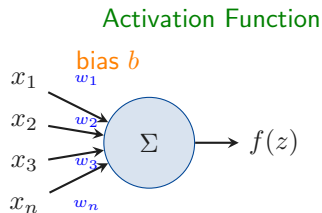
The Neuron: Building Block

Artificial Neuron

- **Inputs** – $x_1, x_2, \dots, x_n$
- **Weights** – $w_1, w_2, \dots, w_n$
- **Bias** – b
- **Activation** – $f(z)$

$$z = \sum_{i=1}^n w_i x_i + b$$

$$\text{Output} = f(z)$$



Activation Functions

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Output: (0, 1)
- Vanishing gradient
- Binary classification

ReLU

$$f(x) = \max(0, x)$$

- Output: $[0, \infty)$
- Most popular
- Solves vanishing gradient

Tanh

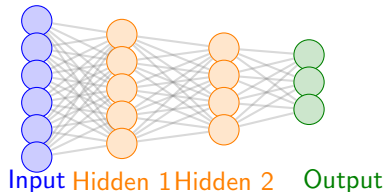
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Output: (-1, 1)
- Zero-centered
- Hidden layers

Neural Network Architecture

Network Components

- **Input Layer** – Receives data
- **Hidden Layers** – Extract features
 - Number of layers = depth
 - Number of neurons = width
- **Output Layer** – Makes predictions
- **Weights & Biases** – Learnable parameters



Training Deep Networks

Training Process

Training Pipeline

1. **Forward Pass** – Compute predictions
2. **Loss Calculation** – Measure error
3. **Backward Pass** – Compute gradients
4. **Weight Update** – Optimize parameters
5. **Iterate** – Repeat until convergence

Key Components

- **Loss Function**
 - Cross-Entropy
 - Mean Squared Error
- **Optimizer**
 - SGD, Adam, RMSprop
- **Learning Rate**
 - Controls step size
- **Epochs**
 - Passes through data
- **Batch Size**
 - Samples per update

Forward and Backward Propagation

Forward Propagation

- Data flows forward
- Each neuron computes:
 - $z = Wx + b$
 - $a = f(z)$
- Output is prediction

Backward Propagation

- Error flows backward
- Compute gradients
- Update weights using:
 - $\Delta W = -\eta \frac{\partial L}{\partial W}$

Forward Propagation

→ Forward



Backward ←

Backward Propagation

PyTorch Fundamentals

PyTorch Overview

What is PyTorch?

- Open-source deep learning framework
- Developed by Meta
- Dynamic computation graphs
- Python-first design
- Industry & research standard

Key Features

- **Tensors** – Like NumPy arrays
- **Autograd** – Automatic differentiation
- **nn.Module** – Build neural networks
- **GPU Support** – CUDA acceleration
- **Ecosystem** – torchvision, torchaudio

PyTorch Basics - Code Example

</> PyTorch Fundamentals

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 # 1. Tensors (like NumPy)
6 x = torch.tensor([1.0, 2.0, 3.0])
7 y = torch.randn(3, 4) # Random tensor
8 z = x + y # GPU-ready operations
9
10 # 2. Autograd (Automatic Differentiation)
11 x = torch.tensor([2.0], requires_grad=True)
12 y = x ** 2
13 y.backward() # Compute gradient
14 print(x.grad) # Output: tensor([4.])
```

PyTorch Basics - Code Example

</> PyTorch Fundamentals

```
1  # 3. Building a Neural Network
2  class SimpleNN(nn.Module):
3      def __init__(self, input_size, hidden_size, output_size):
4          super().__init__()
5          self.fc1 = nn.Linear(input_size, hidden_size)
6          self.fc2 = nn.Linear(hidden_size, output_size)
7          self.relu = nn.ReLU()
8
9      def forward(self, x):
10         x = self.relu(self.fc1(x))
11         x = self.fc2(x)
12         return x
```

PyTorch Basics - Code Example

`</>` PyTorch Fundamentals

```
1  # 4. Training Loop
2  model = SimpleNN(10, 20, 1)
3  criterion = nn.MSELoss()
4  optimizer = optim.Adam(model.parameters(), lr=0.001)
5
6  for epoch in range(100):
7      # Forward pass
8      outputs = model(inputs)
9      loss = criterion(outputs, targets)
10
11     # Backward pass
12     optimizer.zero_grad()
13     loss.backward()
14     optimizer.step()
```

Building Neural Networks

Building a Neural Network in PyTorch

Step-by-Step

1. Define Architecture

- Number of layers
- Activation functions

2. Define Loss Function

3. Define Optimizer

4. Training Loop

5. Evaluation

</> Complete Example

```
1 import torch.nn as nn
2 import torch.optim as optim
3
4 # Define model
5 class ElectionPredictor(nn.Module):
6     def __init__(self):
7         super().__init__()
8         self.fc1 = nn.Linear(5, 64)
9         self.fc2 = nn.Linear(64, 32)
10        self.fc3 = nn.Linear(32, 1)
11        self.relu = nn.ReLU()
12        self.dropout = nn.Dropout(0.2)
13
14    def forward(self, x):
15        x = self.relu(self.fc1(x))
16        x = self.dropout(x)
17        x = self.relu(self.fc2(x))
18        x = self.fc3(x)
19        return x
```

Building a Neural Network in PyTorch

Step-by-Step

1. Define Architecture

- Number of layers
- Activation functions

2. Define Loss Function

3. Define Optimizer

4. Training Loop

5. Evaluation

</> Complete Example

```
1 # Initialize
2 model = ElectionPredictor()
3 criterion = nn.MSELoss()
4 optimizer = optim.Adam(model.parameters(), lr
5                             =0.001)
6
7 # Training loop
8 for epoch in range(100):
9     model.train()
10    optimizer.zero_grad()
11    outputs = model(X_train)
12    loss = criterion(outputs, y_train)
13    loss.backward()
14    optimizer.step()
15
16    if epoch % 10 == 0:
17        print(f"Epoch {epoch}: Loss = {loss.item()
18                :.4f}")
```

Transfer Learning

Transfer Learning

What is Transfer Learning?

- Use pre-trained models
- Fine-tune on new tasks
- Requires less data
- Faster training
- Better performance

Common Pre-trained Models

- **Computer Vision**
 - ResNet
 - VGG
 - EfficientNet
 - YOLO
- **NLP**
 - BERT
 - GPT
 - RoBERTa

Transfer Learning - Code Example

</> Transfer Learning with torchvision

```
1 import torchvision.models as models
2 import torch.nn as nn
3
4 # Load pre-trained ResNet
5 resnet = models.resnet18(pretrained=True)
6
7 # Freeze all layers
8 for param in resnet.parameters():
9     param.requires_grad = False
10
11 # Replace final layer for new task
12 num_classes = 3 # e.g., Low, Medium, High turnout
13 resnet.fc = nn.Linear(resnet.fc.in_features, num_classes)
14
15 # Only train the new layer
16 params_to_train = resnet.fc.parameters()
17
18 # Fine-tune
19 criterion = nn.CrossEntropyLoss()
20 optimizer = optim.Adam(params_to_train, lr=0.001)
```

Transfer Learning - Code Example

</> Transfer Learning with torchvision

```
1 # Training loop (on new data)
2 for epoch in range(50):
3     model.train()
4     outputs = model(images)
5     loss = criterion(outputs, labels)
6     loss.backward()
7     optimizer.step()
```

Model Deployment

Model Deployment Options

Cloud Deployment

- AWS SageMaker
- Google Vertex AI
- Azure ML
- GCP AI Platform

Pros:

- Scalable
- Managed
- Built-in monitoring

On-Premise

- Local servers
- Virtual machines
- Kubernetes

Pros:

- Full control
- Data privacy
- No cloud costs

Edge Deployment

- Mobile devices
- IoT devices
- On-device inference

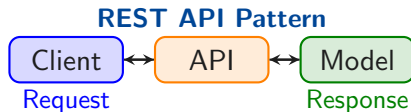
Pros:

- Low latency
- Offline capability
- Privacy-preserving

Deployment Architecture Patterns

Common Patterns

- **REST API** – HTTP endpoints
- **Event-Driven** – Triggers (Kafka, SQS)
- **Batch** – Scheduled predictions
- **Streaming** – Real-time processing



FastAPI Deployment

FastAPI: Modern API Framework

What is FastAPI?

- Modern Python web framework
- Built on Starlette & Pydantic
- Async support
- OpenAPI & Swagger UI built-in

Key Features

- **Speed** – Very high performance
- **Type Hints** – Full IDE support
- **Data Validation** – Pydantic
- **Async/Await** – Non-blocking I/O
- **Automatic Docs** – Swagger UI

✓ Why FastAPI for ML?

- ✓ Lightweight – Perfect for model serving
- ✓ Fast – Low latency predictions
- ✓ Type-Safe – Validation before inference
- ✓ Async – Concurrent predictions
- ✓ Production-Ready – Built-in deployment features

FastAPI ML Deployment - Complete Example

</> Full ML API Service

```
1 from fastapi import FastAPI, HTTPException
2 from pydantic import BaseModel
3 import torch
4 import torch.nn as nn
5 import uvicorn
6
7 # Define model class
8 class ElectionPredictor(nn.Module):
9     def __init__(self):
10         super().__init__()
11         self.fc1 = nn.Linear(5, 64)
12         self.fc2 = nn.Linear(64, 32)
13         self.fc3 = nn.Linear(32, 1)
14         self.relu = nn.ReLU()
15
16     def forward(self, x):
17         x = self.relu(self.fc1(x))
18         x = self.relu(self.fc2(x))
19         return self.fc3(x)
```

FastAPI ML Deployment - Complete Example

</> Full ML API Service

```
1  # Initialize app
2  app = FastAPI(
3      title="Election ML Predictor",
4      description="Predict voter turnout and anomalies",
5      version="1.0.0"
6  )
7
8  # Load model
9  model = ElectionPredictor()
10 model.load_state_dict(torch.load('model.pth'))
11 model.eval()
12
13 # Define schemas
14 class PredictionRequest(BaseModel):
15     registered_voters: int
16     ward: int
17     invalid_votes: int
18     total_votes: int
19
20 class PredictionResponse(BaseModel):
21     turnout_prediction: float
22     confidence: float
23     status: str
```

FastAPI ML Deployment - Complete Example

</> Full ML API Service

```
1 @app.get("/")
2 async def root():
3     return {"message": "Election ML API is running"}
4
5 @app.get("/health")
6 async def health_check():
7     return {"status": "healthy"}
8
9 @app.post("/predict", response_model=PredictionResponse)
10 async def predict(request: PredictionRequest):
11     try:
12         # Prepare features
13         features = torch.tensor([[
14             request.registered_voters,
15             request.ward,
16             request.invalid_votes,
17             request.total_votes,
18             request.invalid_votes / request.total_votes
19         ]], dtype=torch.float32)
20
21         # Make prediction
22         with torch.no_grad():
23             prediction = model(features)
24             confidence = 0.95 # Example
```

FastAPI ML Deployment - Complete Example

</> Full ML API Service

```
1
2     return PredictionResponse(
3         turnout_prediction=float(prediction[0]),
4         confidence=confidence,
5         status="success"
6     )
7 except Exception as e:
8     raise HTTPException(status_code=500, detail=str(e))
9
10 if __name__ == "__main__":
11     uvicorn.run(app, host="0.0.0.0", port=8000)
```

Docker Deployment

Containerization with Docker

Dockerfile

```
1 # Dockerfile
2 FROM pytorch/pytorch:latest
3
4 WORKDIR /app
5
6 COPY requirements.txt .
7 RUN pip install -r requirements.txt
8
9 COPY . .
10
11 EXPOSE 8000
12
13 CMD ["uvicorn", "main:app",
14     "--host", "0.0.0.0",
15     "--port", "8000"]
```

Docker Commands

- Build

```
1 docker build -t election-api .
```

- Run

```
1 docker run -p 8000:8000 election-api
```

- Test

```
1 curl -X POST http://localhost:8000/predict \
2     -H "Content-Type: application/json" \
3     -d '{"registered_voters": 1500, "ward": 5,
4         "invalid_votes": 50, "total_votes":
5         1200}'
```

Deployment Best Practices

Deployment Best Practices

Key Practices

- **Monitoring** – Track model performance
- **Logging** – Log all requests
- **Health Checks** – Status monitoring
- **Versioning** – Model version control
- **A/B Testing** – Compare model versions
- **Rollback** – Quick revert capability

Security Considerations

- API Authentication
- Rate Limiting
- Input Validation
- Data Encryption
- Access Control
- Audit Logging

Remember: Deploying a model is not the end. **Monitor, Maintain, and Iterate**

INEC Applications

Deep Learning for Election Management

INEC Use Cases

- **EC8A Tamper Detection**
 - CNN for document verification
 - Pattern recognition
- **Result Anomaly Detection**
 - LSTM for time series
 - Unsupervised learning
- **Logistics Optimization**
 - RNN for route planning
 - Predictive analytics

Implementation Benefits

- **Accuracy** – State-of-the-art performance
- **Automation** – Reduce manual effort
- **Scalability** – Handle large volumes
- **Real-time** – Instant predictions
- **Cost Savings** – Efficient operations

Key Takeaways

Key Takeaways

What We Covered

1. **Deep Learning** – Neural networks and architectures
2. **PyTorch** – Building and training models
3. **Transfer Learning** – Using pre-trained models
4. **Deployment** – FastAPI and Docker
5. **INEC Applications** – Real-world use cases

💡 Key Principles

- ✓ **Data Quality** – Garbage in, garbage out
- ✓ **Model Selection** – Choose appropriate architecture
- ✓ **Training** – Monitor loss and accuracy
- ✓ **Evaluation** – Test on unseen data
- ✓ **Deployment** – Monitor and maintain

Resources for Further Learning

Online Courses

-  DeepLearning.AI (Coursera)
-  FastAI: Practical Deep Learning
-  PyTorch Official Tutorials

Books

-  *Deep Learning* – Goodfellow, Bengio
-  *PyTorch Deep Learning*
-  *Hands-On ML with Scikit-Learn*

Tools & Libraries

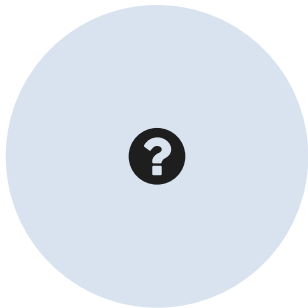
-  PyTorch – Deep learning
-  FastAPI – API deployment
-  Docker – Containerization
-  AWS/GCP/Azure – Cloud deployment

Communities

-  PyTorch Forums
-  Kaggle Competitions

Thank You

Questions & Discussion



"Deep learning is the engine of modern AI innovation."